BÀI THỰC HÀNH 4

SUPABASE — BACKEND-AS-A-SERVICE & AUTHENTICATION

Môn học: Các công nghệ mới trong phát triển phần mềm **Buổi:** 4/8 **Thời lượng:** 4 tiết (180 phút)

Mục tiêu bài thực hành

Sau khi hoàn thành bài thực hành này, sinh viên có khả năng:

1. Hiểu khái niệm Backend-as-a-Service (BaaS) và so sánh Supabase với các giải pháp khác
2. Thiết kế database schema và tạo tables trên Supabase Dashboard
3. Áp dụng Row Level Security (RLS) để bảo vệ dữ liệu ở cấp độ database
4. Tích hợp Supabase Authentication với Next.JS (Email/Password và OAuth)
5. Thực hiện các thao tác CRUD từ Next.JS thông qua Supabase Client SDK
6. Xây dựng ứng dụng full-stack với xác thực và phân quyền hoàn chỉnh

---

Yêu cầu đầu vào

Kiến thức nền:

- Kiến thức React cơ bản: component, props, state, hooks (useState, useEffect)
- TypeScript cơ bản: interface, types
- Hiểu HTTP methods: GET, POST, PUT, DELETE
- Biết sử dụng terminal và Git workflow cơ bản
- Khái niệm cơ bản về cơ sở dữ liệu quan hệ (tables, relationships)

Phần cứng & phần mềm:

- Máy tính đã cài đặt Node.js (phiên bản 18+), VS Code, Git
- Kết nối Internet ổn định

Tài khoản cần chuẩn bị:

- Tài khoản GitHub (đã đăng ký)

• Tài khoản Supabase (đăng ký miễn phí tại supabase.com (https://supabase.com))

Quy ước ký hiệu

Ký hiệu	Ý nghĩa
💡	Mẹo hữu ích
⚠️	Lưu ý quan trọng
📝	Bài tập tự làm
🔍	Giải thích thêm
✅	Kết quả mong đợi

Giới thiệu: Xây dựng ứng dụng Simple Blog

Trong bài thực hành này, chúng ta sẽ xây dựng một **ứng dụng Blog cá nhân (Simple Blog)** hoàn chỉnh — một ứng dụng web cho phép người dùng đăng ký tài khoản, đăng nhập, viết bài, chỉnh sửa và xóa bài viết của mình. Đây là project **mới hoàn toàn**, giúp bạn làm quen với Supabase từ đầu.

Tính năng	Mô tả	Kiến thức áp dụng
Trang chủ	Hiển thị danh sách bài viết công khai	Server-side Data Fetching, Supabase Query
Chi tiết bài viết	Nội dung đầy đủ + bình luận	Dynamic Routes, RLS
Đăng ký / Đăng nhập	Tạo tài khoản, xác thực	Supabase Auth
OAuth	Đăng nhập bằng GitHub	OAuth Integration
Dashboard	Quản lý bài viết cá nhân	Protected Routes, CRUD
Viết / Sửa bài	Form tạo và chỉnh sửa bài viết	Server Actions, RLS Policies
Bình luận	Để lại bình luận trên bài viết	Realtime (Bonus)

Cấu trúc URL của ứng dụng:

Trang	URL	Mô tả	Bảo vệ
Trang chủ	`/`	Danh sách bài viết công khai	Public
Chi tiết bài viết	`/posts/[id]`	Nội dung bài viết + bình luận	Public
Đăng nhập	`/login`	Form đăng nhập	Public
Đăng ký	`/register`	Form đăng ký	Public
Dashboard	`/dashboard`	Quản lý bài viết	🔒 Protected

		của user	
Tạo bài viết	`/dashboard/new`	Form tạo bài mới	🔒 Protected
Chỉnh sửa	`/dashboard/edit/[id]`	Form chỉnh sửa bài viết	🔒 Protected

Phần 1: Giới thiệu Supabase & Khởi tạo Project

⏱️ *Thời lượng: ~30 phút*

Trong phần này, chúng ta sẽ tìm hiểu về Backend-as-a-Service, làm quen với Supabase, và thiết lập project Next.JS mới với Supabase Client SDK.

1.1. Backend-as-a-Service (BaaS) là gì?

Backend-as-a-Service (BaaS) là mô hình dịch vụ đám mây cung cấp sẵn các tính năng backend phổ biến, cho phép developer tập trung vào frontend và business logic mà không cần xây dựng backend từ đầu.

Các tính năng thường có trong BaaS:

Tính năng	Mô tả
Database	Lưu trữ dữ liệu (SQL hoặc NoSQL)
Authentication	Đăng ký, đăng nhập, OAuth, MFA
Authorization	Phân quyền, kiểm soát truy cập
File Storage	Lưu trữ và phục vụ files (images, documents)
Realtime	Cập nhật dữ liệu theo thời gian thực
Serverless Functions	Chạy code backend không cần server
API Auto-generation	Tự động tạo REST/GraphQL API từ database

Ưu điểm của BaaS:

- ✅ **Phát triển nhanh:** Không cần viết backend từ đầu
- ✅ **Tiết kiệm chi phí:** Không cần thuê DevOps, quản lý server
- ✅ **Bảo mật tích hợp:** Authentication, authorization đã được xây dựng sẵn
- ✅ **Khả năng mở rộng:** Scale tự động theo nhu cầu

Nhược điểm:

- ⚠️ **Vendor lock-in:** Phụ thuộc vào nhà cung cấp
- ⚠️ **Giới hạn tùy chỉnh:** Một số use case phức tạp có thể không hỗ trợ
- ⚠️ **Chi phí tăng theo usage:** Có thể đắt khi scale lớn

1.2. So sánh các nền tảng BaaS phổ biến

Tiêu chí	Supabase	Firebase	Appwrite	PocketBase
Database	PostgreSQL (SQL)	Firestore (NoSQL)	MariaDB	SQLite
Open Source	✅ Có	❌ Không	✅ Có	✅ Có
Self-host	✅ Có	❌ Không	✅ Có	✅ Có
Realtime	✅ Có	✅ Có	✅ Có	✅ Có
Auth	✅ Đầy đủ	✅ Đầy đủ	✅ Đầy đủ	✅ Cơ bản
Storage	✅ Có	✅ Có	✅ Có	✅ Có
Edge Functions	✅ Deno	✅ Node.js	✅ Có	❌ Không
Free tier	Rộng rãi	Rộng rãi	Rộng rãi	Miễn phí

🔍 **Tại sao chọn Supabase?**

1. **PostgreSQL:** Database quan hệ mạnh mẽ, quen thuộc với hầu hết developers
2. **Open Source:** Có thể self-host, không bị lock-in hoàn toàn
3. **Row Level Security:** Bảo mật cấp độ database, rất mạnh mẽ
4. **Tích hợp tốt với Next.JS:** SDK và helpers chính thức
5. **Dashboard trực quan:** Dễ quản lý database, auth, storage

---

1.3. Kiến trúc Supabase

Supabase được xây dựng từ các công nghệ open source:

```
┌──────────────────────────────────────────────────────────┐
│                      SUPABASE PLATFORM                   │
├──────────────────────────────────────────────────────────┤
│                                                          │
│   ┌──────────────┐   ┌──────────────┐   ┌──────────────┐
│   │   GoTrue     │   │   PostgREST  │   │   Realtime   │
│   │   (Auth)     │   │   (API)      │   │   (WebSocket)│
│   └──────────────┘   └──────────────┘   └──────────────┘
│
│   ┌──────────────┐   ┌──────────────┐   ┌──────────────┐
│   │   Storage    │   │   Edge       │   │   pgvector   │
│   │   (S3)       │   │   Functions  │   │   (AI Embeddings)
```

```
│   └──────────────┘   └──────────────┘   └──────────────────────┘   │
│                                                                     │
│   ┌─────────────────────────────────────────────────────────────┐   │
│   │                          PostgreSQL                         │   │
│   │              (Database + Row Level Security)                │   │
│   └─────────────────────────────────────────────────────────────┘   │
│                                                                     │
└─────────────────────────────────────────────────────────────────────┘
```

Component	Công nghệ	Chức năng
Database	PostgreSQL	Lưu trữ dữ liệu, RLS, triggers, functions
Auth	GoTrue	Xác thực users, OAuth, JWT tokens
API	PostgREST	Auto-generate REST API từ database
Realtime	Elixir/Phoenix	WebSocket subscriptions, presence
Storage	S3-compatible	Lưu trữ files, images, documents
Edge Functions	Deno	Serverless functions
pgvector	PostgreSQL extension	AI embeddings, vector search

1.4. Tạo project Supabase

Bước 1: Truy cập supabase.com (https://supabase.com) và đăng nhập (hoặc đăng ký nếu chưa có tài khoản).

Bước 2: Từ Dashboard, click **"New Project"**.

Bước 3: Điền thông tin project:

Field	Giá trị
Project name	`simple-blog`
Database Password	Tạo password mạnh (lưu lại để dùng sau)
Region	Chọn region gần nhất (Singapore hoặc Tokyo)
Pricing Plan	Free tier

Bước 4: Click **"Create new project"** và đợi 1-2 phút để Supabase provision database.

✅ **Kết quả:** Project được tạo, bạn sẽ thấy Dashboard với các thông tin:

- Project URL: `https://xxxxx.supabase.co`

- API Key (anon/public): Dùng cho client-side
- API Key (service_role): Dùng cho server-side (giữ bí mật!)

⚠️ **Quan trọng:** Lưu lại **Project URL** và **anon key** — chúng ta sẽ dùng trong bước tiếp theo.

1.5. Khởi tạo project Next.JS

Mở terminal và chạy các lệnh sau:

```bash
# Tạo project Next.JS mới
npx create-next-app@latest simple-blog

# Trả lời các câu hỏi:
# ✓ Would you like to use TypeScript? → Yes
# ✓ Would you like to use ESLint? → Yes
# ✓ Would you like to use Tailwind CSS? → Yes
# ✓ Would you like your code inside a `src/` directory? → Yes
# ✓ Would you like to use App Router? → Yes
# ✓ Would you like to use Turbopack? → Yes
# ✓ Would you like to customize the import alias? → No

# Di chuyển vào thư mục project
cd simple-blog

# Cài đặt Supabase packages
npm install @supabase/supabase-js @supabase/ssr
```

🔍 **Giải thích packages:**

Package	Mục đích
`@supabase/supabase-js`	Supabase JavaScript Client SDK
`@supabase/ssr`	Helpers cho Server-Side Rendering (Next.JS)

1.6. Cấu hình biến môi trường

Tạo file `.env.local` ở thư mục gốc của project:

```bash
# Supabase Configuration
NEXT_PUBLIC_SUPABASE_URL=https://your-project-id.supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY=your-anon-key-here
```

⚠️ **Thay thế** `your-project-id` và `your-anon-key-here` bằng giá trị thực từ Supabase Dashboard.

💡 **Cách lấy credentials:**

1. Vào Supabase Dashboard → Project của bạn
2. Click **"Project Settings"** (biểu tượng bánh răng) ở sidebar
3. Chọn **"API"** trong menu
4. Copy **Project URL** và **anon public** key

🔍 **Tại sao dùng `NEXT_PUBLIC_` prefix?**

Trong Next.JS, biến môi trường có prefix `NEXT_PUBLIC_` sẽ được expose ra client-side (browser). Vì `anon key` được thiết kế để dùng ở client (với RLS bảo vệ), nên chúng ta cần prefix này.

1.7. Tạo Supabase Client Utilities

Supabase cần 2 loại client khác nhau cho Server và Client components trong Next.JS App Router.

Tạo thư mục và files:

```bash
mkdir -p src/lib/supabase
touch src/lib/supabase/client.ts
touch src/lib/supabase/server.ts
touch src/lib/supabase/middleware.ts
```

File `src/lib/supabase/client.ts` — Client cho Browser (Client Components):

```typescript
import { createBrowserClient } from '@supabase/ssr'

export function createClient() {
  return createBrowserClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!
  )
}
```

File `src/lib/supabase/server.ts` — Client cho Server (Server Components, Server Actions, Route Handlers):

[typescript]

```
import { createServerClient } from '@supabase/ssr'
import { cookies } from 'next/headers'

export async function createClient() {
  const cookieStore = await cookies()

  return createServerClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!,
    {
      cookies: {
        getAll() {
          return cookieStore.getAll()
        },
        setAll(cookiesToSet) {
          try {
            cookiesToSet.forEach(({ name, value, options }) =>
              cookieStore.set(name, value, options)
            )
          } catch {
            // The `setAll` method was called from a Server Component.
            // This can be ignored if you have middleware refreshing
            // user sessions.
          }
        },
      },
    }
  )
}
```

File `src/lib/supabase/middleware.ts` — Utility cho Middleware (refresh session):

[typescript]

```
import { createServerClient } from '@supabase/ssr'
import { NextResponse, type NextRequest } from 'next/server'

export async function updateSession(request: NextRequest) {
  let supabaseResponse = NextResponse.next({
    request,
  })

  const supabase = createServerClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!,
    {
      cookies: {
        getAll() {
          return request.cookies.getAll()
        },
        setAll(cookiesToSet) {
          cookiesToSet.forEach(({ name, value, options }) =>
request.cookies.set(name, value))
          supabaseResponse = NextResponse.next({
```

```
            request,
          })
          cookiesToSet.forEach(({ name, value, options }) =>
            supabaseResponse.cookies.set(name, value, options)
          )
        },
      },
    }
  )

  // IMPORTANT: Avoid writing any logic between createServerClient and
  // supabase.auth.getUser(). A simple mistake could make it very hard
to debug
  // issues with users being randomly logged out.

  const {
    data: { user },
  } = await supabase.auth.getUser()

  // Refresh session if expired - required for Server Components
  // https://supabase.com/docs/guides/auth/server-side/nextjs

  return supabaseResponse
}
```

Tạo file `src/middleware.ts` — Next.JS Middleware:

[typescript]

```
import { type NextRequest } from 'next/server'
import { updateSession } from '@/lib/supabase/middleware'

export async function middleware(request: NextRequest) {
  return await updateSession(request)
}

export const config = {
  matcher: [
    /*
     * Match all request paths except for the ones starting with:
     * - _next/static (static files)
     * - _next/image (image optimization files)
     * - favicon.ico (favicon file)
     * Feel free to modify this pattern to include more paths.
     */

'/((?!_next/static|_next/image|favicon.ico|.*\\.(?:svg|png|jpg|jpeg|gif|
webp)$).*)',
  ],
}
```

🔍 Tại sao cần Middleware?

Supabase Auth sử dụng JWT tokens với thời hạn ngắn. Middleware tự động refresh token khi gần hết hạn, đảm bảo user không bị đăng xuất đột ngột.

1.8. Kiểm tra cấu hình

Cập nhật file `src/app/page.tsx` để test kết nối:

[tsx]
```
import { createClient } from '@/lib/supabase/server'

export default async function Home() {
  const supabase = await createClient()

  // Test connection by getting session
  const { data: { session } } = await supabase.auth.getSession()

  return (
    <main className="min-h-screen p-8">
      <h1 className="text-3xl font-bold mb-4">Simple Blog</h1>
      <p className="text-gray-600">
        Supabase connection: {session ? '✅ Logged in' : '⚪ Not logged
in (OK)'}
      </p>
      <p className="text-sm text-gray-400 mt-2">
        Nếu bạn thấy dòng này, Supabase đã được cấu hình thành công!
      </p>
    </main>
  )
}
```

Chạy development server:

[bash]
```
npm run dev
```

Truy cập http://localhost:3000 (http://localhost:3000).

✅ **Kết quả mong đợi:** Trang hiển thị "Supabase connection: ⚪ Not logged in (OK)" — điều này nghĩa là kết nối thành công (chưa đăng nhập là bình thường).

📝 **Bài tập 1.1:** Tự mình thực hiện tất cả các bước từ 1.4 đến 1.8. Chụp screenshot trang web hiển thị thành công.

Phần 2: Database Design & Schema

⏱️ *Thời lượng: ~35 phút*

Trong phần này, chúng ta sẽ thiết kế và tạo database schema cho ứng dụng Blog, bao gồm các bảng `profiles`, `posts`, và `comments`.

2.1. Phân tích yêu cầu và thiết kế schema

Yêu cầu chức năng:

1. Users có thể đăng ký, đăng nhập
2. Mỗi user có profile riêng (avatar, display name)
3. Users có thể tạo, sửa, xóa bài viết của mình
4. Bài viết có thể là draft (nháp) hoặc published (công khai)
5. Visitors có thể xem bài viết công khai
6. Users có thể bình luận trên bài viết

Entity Relationship Diagram (ERD):

```
┌──────────────────────┐        ┌──────────────────────┐
┌──────────────────────┐
│    auth.users        │        │      profiles        │        │
posts          │
│  (Supabase managed)  │        │                      │        │
│
├──────────────────────┤        ├──────────────────────┤
├──────────────────────┤
│  id (UUID) PK        │──────<│ id (UUID) PK, FK     │        │ id (UUID)
PK         │
│  email               │        │ display_name         │        │ title
│
│  ...                 │        │ avatar_url           │    ┌──<│ author_id
(FK)       │
└──────────────────────┘        │ created_at           │    │   │ content
│
                                │ updated_at           │    │   │ excerpt
│
                                └──────────────────────┘    │   │ status
│
                                          │                 │   │ created_at
│
                                          └─────────────────┘   │ updated_at
│
└──────────────────────┘
                                                                │
```

```
┌──────────────────────┐
│      comments        │
├──────────────────────┤
│ id (UUID) PK         │
│ post_id (FK)         │──────────────────<┘
│ author_id (FK)       │
│ content              │
│ created_at           │
└──────────────────────┘
```

2.2. Tạo bảng `profiles`

Supabase Auth tự động tạo bảng `auth.users`, nhưng bảng này có schema cố định. Chúng ta tạo bảng `profiles` riêng để lưu thông tin bổ sung.

Vào Supabase Dashboard → SQL Editor → New query:

[sql]

```sql
-- Tạo bảng profiles
create table public.profiles (
  id uuid not null references auth.users on delete cascade,
  display_name text,
  avatar_url text,
  created_at timestamp with time zone default timezone('utc'::text,
now()) not null,
  updated_at timestamp with time zone default timezone('utc'::text,
now()) not null,

  primary key (id)
);

-- Thêm comment mô tả bảng
comment on table public.profiles is 'User profiles - extends
auth.users';

-- Tạo index cho các trường thường query
create index profiles_display_name_idx on public.profiles
(display_name);
```

🔍 **Giải thích:**

Cột	Kiểu	Mô tả
`id`	UUID	Primary key, references `auth.users.id`
`display_name`	TEXT	Tên hiển thị (có thể null)
`avatar_url`	TEXT	URL ảnh đại diện (có thể null)
`created_at`	TIMESTAMPTZ	Thời điểm tạo profile
`updated_at`	TIMESTAMPTZ	Thời điểm cập nhật gần

		nhất

💡 **`on delete cascade`:** Khi user bị xóa khỏi `auth.users`, profile tương ứng cũng tự động bị xóa.

2.3. Tạo trigger tự động tạo profile

Khi user đăng ký, chúng ta muốn tự động tạo profile cho họ:

[sql]

```
-- Function tự động tạo profile khi có user mới
create or replace function public.handle_new_user()
returns trigger
language plpgsql
security definer set search_path = ''
as $$
begin
  insert into public.profiles (id, display_name, avatar_url)
  values (
    new.id,
    new.raw_user_meta_data ->> 'display_name',
    new.raw_user_meta_data ->> 'avatar_url'
  );
  return new;
end;
$$;

-- Trigger chạy sau khi insert vào auth.users
create trigger on_auth_user_created
  after insert on auth.users
  for each row execute procedure public.handle_new_user();
```

🔍 **Giải thích:**

- **`security definer`:** Function chạy với quyền của người tạo (superuser), không bị ảnh hưởng bởi RLS
- **`raw_user_meta_data`:** Dữ liệu metadata được gửi khi đăng ký (ví dụ: display_name từ form)
- **Trigger:** Tự động gọi function khi có user mới

2.4. Tạo bảng `posts`

[sql]

```
-- Tạo enum cho status
create type post_status as enum ('draft', 'published');
```

```
-- Tạo bảng posts
create table public.posts (
  id uuid default gen_random_uuid() not null,
  author_id uuid not null references public.profiles on delete cascade,
  title text not null,
  slug text not null,
  content text,
  excerpt text,
  status post_status default 'draft' not null,
  created_at timestamp with time zone default timezone('utc'::text,
now()) not null,
  updated_at timestamp with time zone default timezone('utc'::text,
now()) not null,
  published_at timestamp with time zone,

  primary key (id),
  unique (slug)
);

-- Comment mô tả bảng
comment on table public.posts is 'Blog posts';

-- Indexes
create index posts_author_id_idx on public.posts (author_id);
create index posts_status_idx on public.posts (status);
create index posts_published_at_idx on public.posts (published_at desc);
create index posts_slug_idx on public.posts (slug);
```

🔍 Giải thích các cột:

Cột	Kiểu	Mô tả
`id`	UUID	Primary key, tự động generate
`author_id`	UUID	Foreign key → profiles.id
`title`	TEXT	Tiêu đề bài viết (bắt buộc)
`slug`	TEXT	URL-friendly identifier (unique)
`content`	TEXT	Nội dung đầy đủ
`excerpt`	TEXT	Tóm tắt ngắn cho danh sách
`status`	ENUM	'draft' hoặc 'published'
`created_at`	TIMESTAMPTZ	Thời điểm tạo
`updated_at`	TIMESTAMPTZ	Thời điểm cập nhật
`published_at`	TIMESTAMPTZ	Thời điểm publish (null nếu draft)

💡 **Slug là gì?** Slug là phiên bản URL-friendly của title. Ví dụ: "Hello World" → "hello-world". Giúp URL đẹp hơn: `/posts/hello-world` thay vì `/posts/123e4567-e89b...`

2.5. Tạo function tự động generate slug

[sql]

```sql
-- Function tạo slug từ title
create or replace function public.generate_slug(title text)
returns text
language plpgsql
as $$
declare
  base_slug text;
  final_slug text;
  counter integer := 0;
begin
  -- Chuyển về lowercase, thay khoảng trắng bằng dấu gạch ngang
  base_slug := lower(regexp_replace(title, '[^a-zA-Z0-9]+', '-', 'g'));
  -- Xóa dấu gạch ngang ở đầu/cuối
  base_slug := trim(both '-' from base_slug);

  final_slug := base_slug;

  -- Kiểm tra slug đã tồn tại chưa, nếu có thì thêm số
  while exists (select 1 from public.posts where slug = final_slug) loop
    counter := counter + 1;
    final_slug := base_slug || '-' || counter;
  end loop;

  return final_slug;
end;
$$;

-- Trigger tự động set slug khi insert
create or replace function public.set_post_slug()
returns trigger
language plpgsql
as $$
begin
  if new.slug is null or new.slug = '' then
    new.slug := public.generate_slug(new.title);
  end if;
  return new;
end;
$$;

create trigger posts_set_slug
  before insert on public.posts
  for each row execute procedure public.set_post_slug();
```

2.6. Tạo bảng `comments`

[sql]

```sql
-- Tạo bảng comments
create table public.comments (
  id uuid default gen_random_uuid() not null,
  post_id uuid not null references public.posts on delete cascade,
  author_id uuid not null references public.profiles on delete cascade,
  content text not null,
  created_at timestamp with time zone default timezone('utc'::text,
now()) not null,

  primary key (id)
);

-- Comment mô tả
comment on table public.comments is 'Comments on blog posts';

-- Indexes
create index comments_post_id_idx on public.comments (post_id);
create index comments_author_id_idx on public.comments (author_id);
create index comments_created_at_idx on public.comments (created_at
desc);
```

2.7. Tạo function cập nhật `updated_at`

[sql]

```sql
-- Function cập nhật updated_at
create or replace function public.update_updated_at()
returns trigger
language plpgsql
as $$
begin
  new.updated_at = timezone('utc'::text, now());
  return new;
end;
$$;

-- Áp dụng trigger cho profiles
create trigger profiles_updated_at
  before update on public.profiles
  for each row execute procedure public.update_updated_at();

-- Áp dụng trigger cho posts
create trigger posts_updated_at
  before update on public.posts
  for each row execute procedure public.update_updated_at();
```

2.8. Kiểm tra schema

Sau khi chạy tất cả SQL ở trên, vào **Table Editor** trong Supabase Dashboard để xác nhận:

✅ Bạn sẽ thấy 3 bảng: `profiles`, `posts`, `comments`

💡 **Mẹo:** Có thể vào **Database** → **Tables** để xem cấu trúc chi tiết và relationships.

📝 **Bài tập 2.1:** Thêm một số dữ liệu mẫu vào các bảng qua Table Editor:

1. Tạo 1-2 profiles (sẽ cần user id từ auth.users sau khi đăng ký)
2. Tạo 2-3 posts với status 'published'

Phần 3: Row Level Security (RLS)

⏱️ *Thời lượng: ~30 phút*

Row Level Security (RLS) là tính năng của PostgreSQL cho phép kiểm soát quyền truy cập ở cấp độ từng dòng dữ liệu. Đây là một trong những điểm mạnh nhất của Supabase.

3.1. Tại sao cần RLS?

Vấn đề: Khi dùng `anon key` từ client (browser), bất kỳ ai cũng có thể gọi API. Nếu không có RLS:

[javascript]

```
// ⚠️ NGUY HIỂM: Ai cũng có thể xóa bài của người khác!
await supabase
  .from('posts')
  .delete()
  .eq('id', 'any-post-id')
```

Giải pháp: RLS tự động thêm điều kiện `WHERE` vào mọi query:

[sql]

```
-- Với RLS, query trên thực tế trở thành:
DELETE FROM posts
WHERE id = 'any-post-id'
  AND author_id = auth.uid()  -- ← RLS tự động thêm
```

🔍 **RLS = Database-level security.** Bất kể client gửi query gì, database sẽ chỉ trả về/cho phép thao tác trên dữ liệu mà user có quyền.

3.2. Bật RLS cho các bảng

[sql]

```
-- Bật RLS cho tất cả bảng
alter table public.profiles enable row level security;
alter table public.posts enable row level security;
alter table public.comments enable row level security;
```

⚠️ **Quan trọng:** Sau khi bật RLS, **KHÔNG** dữ liệu nào có thể truy cập được cho đến khi bạn tạo policies. Đây là "secure by default".

3.3. RLS Policies cho bảng `profiles`

[sql]

```
-- Policy 1: Ai cũng có thể xem profiles
create policy "Profiles are viewable by everyone"
on public.profiles for select
to anon, authenticated
using (true);

-- Policy 2: Users chỉ có thể update profile của chính mình
create policy "Users can update their own profile"
on public.profiles for update
to authenticated
using ((select auth.uid()) = id)
with check ((select auth.uid()) = id);
```

🔍 **Giải thích:**

Policy	Quyền	Điều kiện
Viewable by everyone	SELECT	`true` → không giới hạn
Update own profile	UPDATE	`auth.uid() = id` → chỉ chủ sở hữu

💡 **`(select auth.uid())`:** Cách viết tối ưu hơn `auth.uid()` — PostgreSQL sẽ cache kết quả, query nhanh hơn.

3.4. RLS Policies cho bảng `posts`

[sql]

```
-- Policy 1: Ai cũng có thể xem bài viết đã publish
create policy "Published posts are viewable by everyone"
on public.posts for select
to anon, authenticated
using (status = 'published');

-- Policy 2: Authors có thể xem tất cả bài của mình (kể cả draft)
create policy "Authors can view all their own posts"
on public.posts for select
to authenticated
using ((select auth.uid()) = author_id);

-- Policy 3: Authenticated users có thể tạo bài viết
create policy "Authenticated users can create posts"
on public.posts for insert
to authenticated
with check ((select auth.uid()) = author_id);

-- Policy 4: Authors có thể update bài của mình
create policy "Authors can update their own posts"
on public.posts for update
to authenticated
using ((select auth.uid()) = author_id)
with check ((select auth.uid()) = author_id);

-- Policy 5: Authors có thể xóa bài của mình
create policy "Authors can delete their own posts"
on public.posts for delete
to authenticated
using ((select auth.uid()) = author_id);
```

🔍 **Logic:**

Người dùng	Xem published	Xem draft	Tạo	Sửa	Xóa
Anonymous	✅	❌	❌	❌	❌
User (không phải author)	✅	❌	❌	❌	❌
Author (chủ bài viết)	✅	✅	✅	✅	✅

3.5. RLS Policies cho bảng `comments`

[sql]

```
-- Policy 1: Ai cũng có thể xem comments của bài đã publish
create policy "Comments on published posts are viewable"
```

```
on public.comments for select
to anon, authenticated
using (
  exists (
    select 1 from public.posts
    where posts.id = comments.post_id
    and posts.status = 'published'
  )
);

-- Policy 2: Authenticated users có thể tạo comments
create policy "Authenticated users can create comments"
on public.comments for insert
to authenticated
with check ((select auth.uid()) = author_id);

-- Policy 3: Authors có thể xóa comments của mình
create policy "Users can delete their own comments"
on public.comments for delete
to authenticated
using ((select auth.uid()) = author_id);
```

🔍 **Policy 1 giải thích:** Sử dụng subquery để kiểm tra bài viết cha có status = 'published' không. Điều này ngăn việc xem comments trên bài draft.

3.6. Test RLS Policies

Cách 1: Test qua Supabase Dashboard

1. Vào **SQL Editor**
2. Chạy query với role cụ thể:

[sql]

```
-- Giả lập anonymous user
set role anon;
select * from public.posts;
-- Chỉ thấy bài published

-- Reset về admin
reset role;
```

Cách 2: Test qua code

Sau khi setup auth ở Phần 4, bạn có thể test bằng cách đăng nhập với user khác nhau và thử các thao tác CRUD.

📝 **Bài tập 3.1:**

1. Thử tạo thêm policy cho phép post author xem tất cả comments trên bài của mình (kể cả khi bài là draft)
2. Viết SQL để test policy này

Phần 4: Authentication

⏱️ *Thời lượng: ~40 phút*

Trong phần này, chúng ta sẽ implement đầy đủ authentication flow: đăng ký, đăng nhập bằng email/password, đăng nhập OAuth (GitHub), và quản lý session.

4.1. Authentication flows trong Supabase

Supabase Auth hỗ trợ nhiều phương thức xác thực:

Phương thức	Mô tả	Sử dụng
Email/Password	Đăng ký/đăng nhập truyền thống	Phổ biến nhất
Magic Link	Gửi link đăng nhập qua email	Không cần nhớ password
OAuth	Đăng nhập qua Google, GitHub, etc.	Tiện lợi cho user
Phone/OTP	Xác thực qua SMS	Apps cần số điện thoại

Trong bài này, chúng ta sẽ implement **Email/Password** và **OAuth (GitHub)**.

4.2. Cấu hình Auth trên Supabase

Bước 1: Vào Supabase Dashboard → **Authentication** → **Providers**

Bước 2: Đảm bảo **Email** provider đã được bật (mặc định là bật)

Bước 3: Cấu hình OAuth với GitHub:

1. Bật **GitHub** provider
2. Truy cập github.com/settings/developers (https://github.com/settings/developers)
3. Click **"New OAuth App"**
4. Điền thông tin:
   - **Application name:** Simple Blog
   - **Homepage URL:** `http://localhost:3000`

- **Authorization callback URL:** `https://your-project-id.supabase.co/auth/v1/callback`

5. Sau khi tạo, copy **Client ID** và **Client Secret**
6. Paste vào Supabase Dashboard

⚠️ **Thay `your-project-id`** bằng project ID thực của bạn.

4.3. Tạo trang đăng ký (Register)

Tạo file `src/app/register/page.tsx`:

[tsx]

```
import { RegisterForm } from '@/components/auth/register-form'

export default function RegisterPage() {
  return (
    <div className="min-h-screen flex items-center justify-center bg-
gray-50">
      <div className="max-w-md w-full space-y-8 p-8 bg-white rounded-lg
shadow">
        <div className="text-center">
          <h2 className="text-3xl font-bold">Đăng ký tài khoản</h2>
          <p className="mt-2 text-gray-600">
            Tạo tài khoản để bắt đầu viết blog
          </p>
        </div>
        <RegisterForm />
      </div>
    </div>
  )
}
```

Tạo file `src/components/auth/register-form.tsx`:

[tsx]

```
'use client'

import { useState } from 'react'
import { useRouter } from 'next/navigation'
import { createClient } from '@/lib/supabase/client'
import Link from 'next/link'

export function RegisterForm() {
  const router = useRouter()
  const supabase = createClient()

  const [email, setEmail] = useState('')
  const [password, setPassword] = useState('')
  const [displayName, setDisplayName] = useState('')
  const [error, setError] = useState<string | null>(null)
```

```
  const [loading, setLoading] = useState(false)

  const handleRegister = async (e: React.FormEvent) => {
    e.preventDefault()
    setError(null)
    setLoading(true)

    try {
      const { data, error } = await supabase.auth.signUp({
        email,
        password,
        options: {
          data: {
            display_name: displayName,
          },
        },
      })

      if (error) {
        setError(error.message)
        return
      }

      if (data.user) {
        // Đăng ký thành công
        router.push('/login?message=Đăng ký thành công! Vui lòng kiểm
tra email để xác nhận.')
      }
    } catch (err) {
      setError('Có lỗi xảy ra. Vui lòng thử lại.')
    } finally {
      setLoading(false)
    }
  }

  return (
    <form onSubmit={handleRegister} className="mt-8 space-y-6">
      {error && (
        <div className="bg-red-50 text-red-500 p-3 rounded-md text-sm">
          {error}
        </div>
      )}

      <div className="space-y-4">
        <div>
          <label htmlFor="displayName" className="block text-sm font-
medium text-gray-700">
            Tên hiển thị
          </label>
          <input
            id="displayName"
            type="text"
            value={displayName}
            onChange={(e) => setDisplayName(e.target.value)}
            required
```

```
            className="mt-1 block w-full px-3 py-2 border border-gray-
300 rounded-md shadow-sm focus:outline-none focus:ring-blue-500
focus:border-blue-500"
            placeholder="Nguyễn Văn A"
          />
        </div>

        <div>
          <label htmlFor="email" className="block text-sm font-medium
text-gray-700">
            Email
          </label>
          <input
            id="email"
            type="email"
            value={email}
            onChange={(e) => setEmail(e.target.value)}
            required
            className="mt-1 block w-full px-3 py-2 border border-gray-
300 rounded-md shadow-sm focus:outline-none focus:ring-blue-500
focus:border-blue-500"
            placeholder="email@example.com"
          />
        </div>

        <div>
          <label htmlFor="password" className="block text-sm font-medium
text-gray-700">
            Mật khẩu
          </label>
          <input
            id="password"
            type="password"
            value={password}
            onChange={(e) => setPassword(e.target.value)}
            required
            minLength={6}
            className="mt-1 block w-full px-3 py-2 border border-gray-
300 rounded-md shadow-sm focus:outline-none focus:ring-blue-500
focus:border-blue-500"
            placeholder="••••••••"
          />
          <p className="mt-1 text-xs text-gray-500">Tối thiểu 6 ký
tự</p>
        </div>
      </div>

      <button
        type="submit"
        disabled={loading}
        className="w-full flex justify-center py-2 px-4 border border-
transparent rounded-md shadow-sm text-sm font-medium text-white bg-blue-
600 hover:bg-blue-700 focus:outline-none focus:ring-2 focus:ring-offset-
2 focus:ring-blue-500 disabled:opacity-50 disabled:cursor-not-allowed"
      >
        {loading ? 'Đang xử lý...' : 'Đăng ký'}
```

```
      </button>

      <p className="text-center text-sm text-gray-600">
        Đã có tài khoản?{' '}
        <Link href="/login" className="text-blue-600 hover:text-blue-
500">
          Đăng nhập
        </Link>
      </p>
    </form>
  )
}
```

4.4. Tạo trang đăng nhập (Login)

Tạo file `src/app/login/page.tsx`:

[tsx]

```
import { LoginForm } from '@/components/auth/login-form'

export default function LoginPage({
  searchParams,
}: {
  searchParams: { message?: string }
}) {
  return (
    <div className="min-h-screen flex items-center justify-center bg-
gray-50">
      <div className="max-w-md w-full space-y-8 p-8 bg-white rounded-lg
shadow">
        <div className="text-center">
          <h2 className="text-3xl font-bold">Đăng nhập</h2>
          <p className="mt-2 text-gray-600">
            Đăng nhập để quản lý blog của bạn
          </p>
        </div>

        {searchParams?.message && (
          <div className="bg-green-50 text-green-700 p-3 rounded-md
text-sm">
            {searchParams.message}
          </div>
        )}

        <LoginForm />
      </div>
    </div>
  )
}
```

Tạo file `src/components/auth/login-form.tsx`:

[tsx]

```tsx
'use client'

import { useState } from 'react'
import { useRouter } from 'next/navigation'
import { createClient } from '@/lib/supabase/client'
import Link from 'next/link'

export function LoginForm() {
  const router = useRouter()
  const supabase = createClient()

  const [email, setEmail] = useState('')
  const [password, setPassword] = useState('')
  const [error, setError] = useState<string | null>(null)
  const [loading, setLoading] = useState(false)

  const handleLogin = async (e: React.FormEvent) => {
    e.preventDefault()
    setError(null)
    setLoading(true)

    try {
      const { data, error } = await supabase.auth.signInWithPassword({
        email,
        password,
      })

      if (error) {
        setError(error.message)
        return
      }

      if (data.session) {
        router.push('/dashboard')
        router.refresh()
      }
    } catch (err) {
      setError('Có lỗi xảy ra. Vui lòng thử lại.')
    } finally {
      setLoading(false)
    }
  }

  const handleGitHubLogin = async () => {
    setError(null)

    const { error } = await supabase.auth.signInWithOAuth({
      provider: 'github',
      options: {
        redirectTo: `${window.location.origin}/auth/callback`,
      },
    })
```

```
    if (error) {
      setError(error.message)
    }
  }

  return (
    <div className="mt-8 space-y-6">
      {error && (
        <div className="bg-red-50 text-red-500 p-3 rounded-md text-sm">
          {error}
        </div>
      )}

      {/* OAuth Buttons */}
      <div>
        <button
          type="button"
          onClick={handleGitHubLogin}
          className="w-full flex items-center justify-center gap-2 py-2
px-4 border border-gray-300 rounded-md shadow-sm text-sm font-medium
text-gray-700 bg-white hover:bg-gray-50 focus:outline-none focus:ring-2
focus:ring-offset-2 focus:ring-blue-500"
        >
          <svg className="w-5 h-5" fill="currentColor" viewBox="0 0 24
24">
            <path fillRule="evenodd" d="M12 2C6.477 2 2 6.484 2 12.017c0
4.425 2.865 8.18 6.839 9.504.5.092.682-.217.682-.483 0-.237-.008-.868-
.013-1.703-2.782.605-3.369-1.343-3.369-1.343-.454-1.158-1.11-1.466-1.11-
1.466-.908-.62.069-.608.069-.608 1.003.07 1.531 1.032 1.531 1.032.892
1.53 2.341 1.088 2.91.832.092-.647.35-1.088.636-1.338-2.22-.253-4.555-
1.113-4.555-4.951 0-1.093.39-1.988 1.029-2.688-.103-.253-.446-1.272.098-
2.65 0 0 .84-.27 2.75 1.026A9.564 9.564 0 0112 6.844c.85.004 1.705.115
2.504.337 1.909-1.296 2.747-1.027 2.747-1.027.546 1.379.202 2.398.1
2.651.64.7 1.028 1.595 1.028 2.688 0 3.848-2.339 4.695-4.566
4.943.359.309.678.92.678 1.855 0 1.338-.012 2.419-.012 2.747 0
.268.18.58.688.482A10.019 10.019 0 0022 12.017C22 6.484 17.522 2 12 2z"
clipRule="evenodd" />
          </svg>
          Đăng nhập với GitHub
        </button>
      </div>

      <div className="relative">
        <div className="absolute inset-0 flex items-center">
          <div className="w-full border-t border-gray-300" />
        </div>
        <div className="relative flex justify-center text-sm">
          <span className="px-2 bg-white text-gray-500">Hoặc</span>
        </div>
      </div>

      {/* Email/Password Form */}
      <form onSubmit={handleLogin} className="space-y-4">
        <div>
```

```
          <label htmlFor="email" className="block text-sm font-medium
text-gray-700">
            Email
          </label>
          <input
            id="email"
            type="email"
            value={email}
            onChange={(e) => setEmail(e.target.value)}
            required
            className="mt-1 block w-full px-3 py-2 border border-gray-
300 rounded-md shadow-sm focus:outline-none focus:ring-blue-500
focus:border-blue-500"
            placeholder="email@example.com"
          />
        </div>

        <div>
          <label htmlFor="password" className="block text-sm font-medium
text-gray-700">
            Mật khẩu
          </label>
          <input
            id="password"
            type="password"
            value={password}
            onChange={(e) => setPassword(e.target.value)}
            required
            className="mt-1 block w-full px-3 py-2 border border-gray-
300 rounded-md shadow-sm focus:outline-none focus:ring-blue-500
focus:border-blue-500"
            placeholder="••••••••"
          />
        </div>

        <button
          type="submit"
          disabled={loading}
          className="w-full flex justify-center py-2 px-4 border border-
transparent rounded-md shadow-sm text-sm font-medium text-white bg-blue-
600 hover:bg-blue-700 focus:outline-none focus:ring-2 focus:ring-offset-
2 focus:ring-blue-500 disabled:opacity-50 disabled:cursor-not-allowed"
        >
          {loading ? 'Đang xử lý...' : 'Đăng nhập'}
        </button>
      </form>

      <p className="text-center text-sm text-gray-600">
        Chưa có tài khoản?{' '}
        <Link href="/register" className="text-blue-600 hover:text-blue-
500">
          Đăng ký ngay
        </Link>
      </p>
    </div>
  )
```

```
}
```

4.5. Tạo OAuth Callback Route

Khi user đăng nhập qua GitHub, họ sẽ được redirect về URL callback. Chúng ta cần xử lý callback này:

Tạo file `src/app/auth/callback/route.ts`:

[typescript]

```
import { createClient } from '@/lib/supabase/server'
import { NextResponse } from 'next/server'

export async function GET(request: Request) {
  const requestUrl = new URL(request.url)
  const code = requestUrl.searchParams.get('code')
  const origin = requestUrl.origin

  if (code) {
    const supabase = await createClient()
    await supabase.auth.exchangeCodeForSession(code)
  }

  // Redirect về dashboard sau khi đăng nhập thành công
  return NextResponse.redirect(`${origin}/dashboard`)
}
```

🔍 **Flow OAuth:**

1. User click "Đăng nhập với GitHub"
2. Redirect đến GitHub để xác thực
3. GitHub redirect về `/auth/callback?code=xxx`
4. Route handler exchange code lấy session
5. Redirect về `/dashboard`

4.6. Tạo Server Action cho Logout

Tạo file `src/app/actions/auth.ts`:

[typescript]

```
'use server'

import { createClient } from '@/lib/supabase/server'
import { redirect } from 'next/navigation'
```

```
export async function logout() {
  const supabase = await createClient()
  await supabase.auth.signOut()
  redirect('/login')
}
```

4.7. Bảo vệ Routes với Middleware

Cập nhật file `src/lib/supabase/middleware.ts` để redirect users chưa đăng nhập:

[typescript]

```
import { createServerClient } from '@supabase/ssr'
import { NextResponse, type NextRequest } from 'next/server'

export async function updateSession(request: NextRequest) {
  let supabaseResponse = NextResponse.next({
    request,
  })

  const supabase = createServerClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!,
    {
      cookies: {
        getAll() {
          return request.cookies.getAll()
        },
        setAll(cookiesToSet) {
          cookiesToSet.forEach(({ name, value, options }) =>
request.cookies.set(name, value))
          supabaseResponse = NextResponse.next({
            request,
          })
          cookiesToSet.forEach(({ name, value, options }) =>
            supabaseResponse.cookies.set(name, value, options)
          )
        },
      },
    }
  )

  const {
    data: { user },
  } = await supabase.auth.getUser()

  // Protected routes - redirect to login if not authenticated
  const protectedPaths = ['/dashboard']
  const isProtectedPath = protectedPaths.some(path =>
    request.nextUrl.pathname.startsWith(path)
  )
```

```
    if (isProtectedPath && !user) {
      const url = request.nextUrl.clone()
      url.pathname = '/login'
      return NextResponse.redirect(url)
    }

    // Redirect logged-in users away from auth pages
    const authPaths = ['/login', '/register']
    const isAuthPath = authPaths.some(path =>
      request.nextUrl.pathname === path
    )

    if (isAuthPath && user) {
      const url = request.nextUrl.clone()
      url.pathname = '/dashboard'
      return NextResponse.redirect(url)
    }

    return supabaseResponse
  }
```

4.8. Tạo Header Component với trạng thái đăng nhập

Tạo file `src/components/layout/header.tsx`:

[tsx]

```
  import Link from 'next/link'
  import { createClient } from '@/lib/supabase/server'
  import { logout } from '@/app/actions/auth'

  export async function Header() {
    const supabase = await createClient()
    const { data: { user } } = await supabase.auth.getUser()

    return (
      <header className="bg-white shadow">
        <div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8">
          <div className="flex justify-between items-center h-16">
            <Link href="/" className="text-xl font-bold text-gray-900">
              Simple Blog
            </Link>

            <nav className="flex items-center gap-4">
              <Link href="/" className="text-gray-600 hover:text-gray-
  900">
                Trang chủ
              </Link>

              {user ? (
                <>
                  <Link
                    href="/dashboard"
```

```
                  className="text-gray-600 hover:text-gray-900"
                >
                  Dashboard
                </Link>
                <form action={logout}>
                  <button
                    type="submit"
                    className="text-gray-600 hover:text-gray-900"
                  >
                    Đăng xuất
                  </button>
                </form>
              </>
            ) : (
              <>
                <Link
                  href="/login"
                  className="text-gray-600 hover:text-gray-900"
                >
                  Đăng nhập
                </Link>
                <Link
                  href="/register"
                  className="bg-blue-600 text-white px-4 py-2 rounded-md
hover:bg-blue-700"
                >
                  Đăng ký
                </Link>
              </>
            )}
          </nav>
        </div>
      </div>
    </header>
  )
}
```

Cập nhật `src/app/layout.tsx`:

[tsx]

```
import type { Metadata } from 'next'
import { Inter } from 'next/font/google'
import './globals.css'
import { Header } from '@/components/layout/header'

const inter = Inter({ subsets: ['latin', 'vietnamese'] })

export const metadata: Metadata = {
  title: 'Simple Blog',
  description: 'A simple blog built with Next.JS and Supabase',
}

export default function RootLayout({
  children,
```

```
}: {
  children: React.ReactNode
}) {
  return (
    <html lang="vi">
      <body className={inter.className}>
        <Header />
        {children}
      </body>
    </html>
  )
}
```

4.9. Kiểm tra Authentication

Test flow:

1. Truy cập http://localhost:3000/register (http://localhost:3000/register)
2. Đăng ký tài khoản mới
3. Kiểm tra email (nếu cấu hình email confirmation) hoặc đăng nhập ngay
4. Truy cập http://localhost:3000/login (http://localhost:3000/login)
5. Đăng nhập với email/password
6. Thử đăng nhập với GitHub
7. Sau khi đăng nhập, bạn sẽ được redirect về `/dashboard`
8. Thử truy cập `/dashboard` khi chưa đăng nhập → redirect về `/login`

✅ **Kết quả mong đợi:**

- Đăng ký, đăng nhập thành công
- Header hiển thị đúng trạng thái (Đăng nhập/Đăng xuất)
- Routes được bảo vệ đúng cách

📝 **Bài tập 4.1:** Thêm tính năng "Quên mật khẩu" sử dụng `supabase.auth.resetPasswordForEmail()`.

Phần 5: CRUD Operations — Blog Posts

⏱️ *Thời lượng: ~35 phút*

Trong phần này, chúng ta sẽ implement đầy đủ các thao tác CRUD cho bài viết: tạo, đọc, cập nhật, và xóa.

5.1. Định nghĩa TypeScript Types

Tạo file `src/types/database.ts`:

[typescript]

```
export type PostStatus = 'draft' | 'published'

export interface Profile {
  id: string
  display_name: string | null
  avatar_url: string | null
  created_at: string
  updated_at: string
}

export interface Post {
  id: string
  author_id: string
  title: string
  slug: string
  content: string | null
  excerpt: string | null
  status: PostStatus
  created_at: string
  updated_at: string
  published_at: string | null
  // Joined data
  profiles?: Profile
}

export interface Comment {
  id: string
  post_id: string
  author_id: string
  content: string
  created_at: string
  // Joined data
  profiles?: Profile
}
```

💡 **Tip:** Supabase CLI có thể tự động generate types từ database schema. Xem Supabase CLI docs (https://supabase.com/docs/guides/cli).

5.2. Tạo trang Dashboard

Tạo file `src/app/dashboard/page.tsx`:

[tsx]

```
import Link from 'next/link'
```

```
import { createClient } from '@/lib/supabase/server'
import { redirect } from 'next/navigation'
import { PostList } from '@/components/dashboard/post-list'

export default async function DashboardPage() {
  const supabase = await createClient()

  const { data: { user } } = await supabase.auth.getUser()

  if (!user) {
    redirect('/login')
  }

  // Lấy tất cả bài viết của user (kể cả draft)
  const { data: posts, error } = await supabase
    .from('posts')
    .select('*')
    .eq('author_id', user.id)
    .order('created_at', { ascending: false })

  if (error) {
    console.error('Error fetching posts:', error)
  }

  return (
    <main className="max-w-4xl mx-auto px-4 py-8">
      <div className="flex justify-between items-center mb-8">
        <h1 className="text-3xl font-bold">Bài viết của tôi</h1>
        <Link
          href="/dashboard/new"
          className="bg-blue-600 text-white px-4 py-2 rounded-md
hover:bg-blue-700"
        >
          + Viết bài mới
        </Link>
      </div>

      {posts && posts.length > 0 ? (
        <PostList posts={posts} />
      ) : (
        <div className="text-center py-12 bg-gray-50 rounded-lg">
          <p className="text-gray-500 mb-4">Bạn chưa có bài viết
nào.</p>
          <Link
            href="/dashboard/new"
            className="text-blue-600 hover:text-blue-500"
          >
            Viết bài đầu tiên →
          </Link>
        </div>
      )}
    </main>
  )
}
```

Tạo file `src/components/dashboard/post-list.tsx`:

[tsx]

```
import Link from 'next/link'
import { Post } from '@/types/database'
import { DeletePostButton } from './delete-post-button'

interface PostListProps {
  posts: Post[]
}

export function PostList({ posts }: PostListProps) {
  return (
    <div className="space-y-4">
      {posts.map((post) => (
        <div
          key={post.id}
          className="bg-white p-4 rounded-lg shadow border border-gray-
200"
        >
          <div className="flex justify-between items-start">
            <div className="flex-1">
              <div className="flex items-center gap-2 mb-2">
                <h2 className="text-xl font-semibold">{post.title}</h2>
                <span
                  className={`px-2 py-1 text-xs rounded-full ${
                    post.status === 'published'
                      ? 'bg-green-100 text-green-800'
                      : 'bg-yellow-100 text-yellow-800'
                  }`}
                >
                  {post.status === 'published' ? 'Đã xuất bản' : 'Bản
nháp'}
                </span>
              </div>
              {post.excerpt && (
                <p className="text-gray-600 text-sm mb-
2">{post.excerpt}</p>
              )}
              <p className="text-gray-400 text-xs">
                Tạo ngày: {new
Date(post.created_at).toLocaleDateString('vi-VN')}
              </p>
            </div>

            <div className="flex items-center gap-2">
              <Link
                href={`/posts/${post.slug}`}
                className="text-gray-500 hover:text-gray-700 px-3 py-1
text-sm"
              >
                Xem
              </Link>
              <Link
                href={`/dashboard/edit/${post.id}`}
```

```
                  className="text-blue-600 hover:text-blue-500 px-3 py-1
text-sm"
                >
                  Sửa
                </Link>
                <DeletePostButton postId={post.id} postTitle={post.title}
/>
              </div>
            </div>
          </div>
        ))}
      </div>
    )
  }
```

5.3. Tạo form viết bài mới

Tạo file `src/app/dashboard/new/page.tsx`:

[tsx]

```
import { PostForm } from '@/components/dashboard/post-form'

export default function NewPostPage() {
  return (
    <main className="max-w-4xl mx-auto px-4 py-8">
      <h1 className="text-3xl font-bold mb-8">Viết bài mới</h1>
      <PostForm />
    </main>
  )
}
```

Tạo file `src/components/dashboard/post-form.tsx`:

[tsx]

```
'use client'

import { useState } from 'react'
import { useRouter } from 'next/navigation'
import { createClient } from '@/lib/supabase/client'
import { Post, PostStatus } from '@/types/database'

interface PostFormProps {
  post?: Post
}

export function PostForm({ post }: PostFormProps) {
  const router = useRouter()
  const supabase = createClient()
  const isEditing = !!post

  const [title, setTitle] = useState(post?.title || '')
```

```
  const [content, setContent] = useState(post?.content || '')
  const [excerpt, setExcerpt] = useState(post?.excerpt || '')
  const [status, setStatus] = useState<PostStatus>(post?.status ||
'draft')
  const [error, setError] = useState<string | null>(null)
  const [loading, setLoading] = useState(false)

  const handleSubmit = async (e: React.FormEvent) => {
    e.preventDefault()
    setError(null)
    setLoading(true)

    try {
      const { data: { user } } = await supabase.auth.getUser()

      if (!user) {
        setError('Bạn cần đăng nhập để thực hiện thao tác này')
        return
      }

      const postData = {
        title,
        content,
        excerpt,
        status,
        author_id: user.id,
        published_at: status === 'published' ? new Date().toISOString()
: null,
      }

      if (isEditing) {
        // Update existing post
        const { error } = await supabase
          .from('posts')
          .update(postData)
          .eq('id', post.id)

        if (error) throw error
      } else {
        // Create new post
        const { error } = await supabase
          .from('posts')
          .insert(postData)

        if (error) throw error
      }

      router.push('/dashboard')
      router.refresh()
    } catch (err: any) {
      setError(err.message || 'Có lỗi xảy ra. Vui lòng thử lại.')
    } finally {
      setLoading(false)
    }
  }
```

```
  return (
    <form onSubmit={handleSubmit} className="space-y-6">
      {error && (
        <div className="bg-red-50 text-red-500 p-3 rounded-md text-sm">
          {error}
        </div>
      )}

      <div>
        <label htmlFor="title" className="block text-sm font-medium
text-gray-700">
          Tiêu đề <span className="text-red-500">*</span>
        </label>
        <input
          id="title"
          type="text"
          value={title}
          onChange={(e) => setTitle(e.target.value)}
          required
          className="mt-1 block w-full px-3 py-2 border border-gray-300
rounded-md shadow-sm focus:outline-none focus:ring-blue-500
focus:border-blue-500"
          placeholder="Nhập tiêu đề bài viết"
        />
      </div>

      <div>
        <label htmlFor="excerpt" className="block text-sm font-medium
text-gray-700">
          Tóm tắt
        </label>
        <input
          id="excerpt"
          type="text"
          value={excerpt}
          onChange={(e) => setExcerpt(e.target.value)}
          className="mt-1 block w-full px-3 py-2 border border-gray-300
rounded-md shadow-sm focus:outline-none focus:ring-blue-500
focus:border-blue-500"
          placeholder="Mô tả ngắn về bài viết (hiển thị trong danh
sách)"
        />
      </div>

      <div>
        <label htmlFor="content" className="block text-sm font-medium
text-gray-700">
          Nội dung
        </label>
        <textarea
          id="content"
          value={content}
          onChange={(e) => setContent(e.target.value)}
          rows={15}
```

```
          className="mt-1 block w-full px-3 py-2 border border-gray-300
rounded-md shadow-sm focus:outline-none focus:ring-blue-500
focus:border-blue-500 font-mono"
          placeholder="Viết nội dung bài viết của bạn..."
        />
        <p className="mt-1 text-xs text-gray-500">
          Hỗ trợ Markdown
        </p>
      </div>

      <div>
        <label htmlFor="status" className="block text-sm font-medium
text-gray-700">
          Trạng thái
        </label>
        <select
          id="status"
          value={status}
          onChange={(e) => setStatus(e.target.value as PostStatus)}
          className="mt-1 block w-full px-3 py-2 border border-gray-300
rounded-md shadow-sm focus:outline-none focus:ring-blue-500
focus:border-blue-500"
        >
          <option value="draft">Bản nháp</option>
          <option value="published">Xuất bản</option>
        </select>
      </div>

      <div className="flex justify-end gap-4">
        <button
          type="button"
          onClick={() => router.back()}
          className="px-4 py-2 text-gray-700 hover:text-gray-900"
        >
          Hủy
        </button>
        <button
          type="submit"
          disabled={loading}
          className="px-4 py-2 bg-blue-600 text-white rounded-md
hover:bg-blue-700 disabled:opacity-50 disabled:cursor-not-allowed"
        >
          {loading ? 'Đang lưu...' : isEditing ? 'Cập nhật' : 'Tạo bài
viết'}
        </button>
      </div>
    </form>
  )
}
```

5.4. Tạo trang chỉnh sửa bài viết

Tạo file `src/app/dashboard/edit/[id]/page.tsx`:

[tsx]

```
import { createClient } from '@/lib/supabase/server'
import { redirect, notFound } from 'next/navigation'
import { PostForm } from '@/components/dashboard/post-form'

interface EditPostPageProps {
  params: { id: string }
}

export default async function EditPostPage({ params }:
EditPostPageProps) {
  const supabase = await createClient()

  const { data: { user } } = await supabase.auth.getUser()

  if (!user) {
    redirect('/login')
  }

  const { data: post, error } = await supabase
    .from('posts')
    .select('*')
    .eq('id', params.id)
    .eq('author_id', user.id) // Chỉ cho phép edit bài của mình
    .single()

  if (error || !post) {
    notFound()
  }

  return (
    <main className="max-w-4xl mx-auto px-4 py-8">
      <h1 className="text-3xl font-bold mb-8">Chỉnh sửa bài viết</h1>
      <PostForm post={post} />
    </main>
  )
}
```

5.5. Tạo nút xóa bài viết

Tạo file `src/components/dashboard/delete-post-button.tsx`:

[tsx]

```
'use client'

import { useState } from 'react'
import { useRouter } from 'next/navigation'
import { createClient } from '@/lib/supabase/client'

interface DeletePostButtonProps {
  postId: string
```

```
  postTitle: string
}

export function DeletePostButton({ postId, postTitle }:
DeletePostButtonProps) {
  const router = useRouter()
  const supabase = createClient()
  const [loading, setLoading] = useState(false)

  const handleDelete = async () => {
    const confirmed = window.confirm(
      `Bạn có chắc muốn xóa bài viết "${postTitle}"? Hành động này không
thể hoàn tác.`
    )

    if (!confirmed) return

    setLoading(true)

    try {
      const { error } = await supabase
        .from('posts')
        .delete()
        .eq('id', postId)

      if (error) throw error

      router.refresh()
    } catch (err) {
      alert('Có lỗi xảy ra khi xóa bài viết')
      console.error(err)
    } finally {
      setLoading(false)
    }
  }

  return (
    <button
      onClick={handleDelete}
      disabled={loading}
      className="text-red-600 hover:text-red-500 px-3 py-1 text-sm
disabled:opacity-50"
    >
      {loading ? 'Đang xóa...' : 'Xóa'}
    </button>
  )
}
```

5.6. Tạo trang danh sách bài viết (Public)

Cập nhật file `src/app/page.tsx`:

[tsx]

```
import Link from 'next/link'
import { createClient } from '@/lib/supabase/server'

export default async function HomePage() {
  const supabase = await createClient()

  // Lấy bài viết đã publish, kèm thông tin author
  const { data: posts, error } = await supabase
    .from('posts')
    .select(`
      *,
      profiles (
        display_name,
        avatar_url
      )
    `)
    .eq('status', 'published')
    .order('published_at', { ascending: false })

  if (error) {
    console.error('Error fetching posts:', error)
  }

  return (
    <main className="max-w-4xl mx-auto px-4 py-8">
      <h1 className="text-3xl font-bold mb-8">Bài viết mới nhất</h1>

      {posts && posts.length > 0 ? (
        <div className="space-y-6">
          {posts.map((post) => (
            <article
              key={post.id}
              className="bg-white p-6 rounded-lg shadow border border-
gray-200"
            >
              <Link href={`/posts/${post.slug}`}>
                <h2 className="text-2xl font-semibold hover:text-blue-
600 transition-colors">
                  {post.title}
                </h2>
              </Link>

              {post.excerpt && (
                <p className="text-gray-600 mt-2">{post.excerpt}</p>
              )}

              <div className="flex items-center gap-4 mt-4 text-sm text-
gray-500">
                <span>
                  Bởi {post.profiles?.display_name || 'Ẩn danh'}
                </span>
                <span>•</span>
                <span>
                  {post.published_at
```

```
                    ? new
Date(post.published_at).toLocaleDateString('vi-VN')
                    : 'Chưa xuất bản'}
                </span>
              </div>

              <Link
                href={`/posts/${post.slug}`}
                className="inline-block mt-4 text-blue-600 hover:text-
blue-500"
              >
                Đọc tiếp →
              </Link>
            </article>
          ))}
        </div>
      ) : (
        <div className="text-center py-12 bg-gray-50 rounded-lg">
          <p className="text-gray-500">Chưa có bài viết nào.</p>
        </div>
      )}
    </main>
  )
}
```

5.7. Tạo trang chi tiết bài viết

Tạo file `src/app/posts/[slug]/page.tsx`:

[tsx]

```
import { createClient } from '@/lib/supabase/server'
import { notFound } from 'next/navigation'
import type { Metadata } from 'next'

interface PostPageProps {
  params: { slug: string }
}

export async function generateMetadata({ params }: PostPageProps):
Promise<Metadata> {
  const supabase = await createClient()

  const { data: post } = await supabase
    .from('posts')
    .select('title, excerpt')
    .eq('slug', params.slug)
    .eq('status', 'published')
    .single()

  return {
    title: post?.title || 'Bài viết',
    description: post?.excerpt || '',
```

```
  }
}

export default async function PostPage({ params }: PostPageProps) {
  const supabase = await createClient()

  const { data: post, error } = await supabase
    .from('posts')
    .select(`
      *,
      profiles (
        display_name,
        avatar_url
      )
    `)
    .eq('slug', params.slug)
    .eq('status', 'published')
    .single()

  if (error || !post) {
    notFound()
  }

  return (
    <main className="max-w-3xl mx-auto px-4 py-8">
      <article>
        <header className="mb-8">
          <h1 className="text-4xl font-bold mb-4">{post.title}</h1>

          <div className="flex items-center gap-4 text-gray-500">
            <span>Bởi {post.profiles?.display_name || 'Ẩn danh'}</span>
            <span>•</span>
            <time>
              {post.published_at
                ? new Date(post.published_at).toLocaleDateString('vi-
VN', {
                    year: 'numeric',
                    month: 'long',
                    day: 'numeric',
                  })
                : ''}
            </time>
          </div>
        </header>

        <div className="prose prose-lg max-w-none">
          {/* Render markdown content */}
          {post.content?.split('\n').map((paragraph, index) => (
            <p key={index}>{paragraph}</p>
          ))}
        </div>
      </article>
    </main>
  )
}
```

💡 **Tip:** Để render Markdown đẹp hơn, bạn có thể cài đặt `react-markdown` và `@tailwindcss/typography`:

```bash
npm install react-markdown
npm install -D @tailwindcss/typography
```

5.8. Kiểm tra CRUD

Test các chức năng:

1. **CREATE:** Vào `/dashboard/new`, tạo bài viết mới
2. **READ:** Xem danh sách ở trang chủ `/` và chi tiết `/posts/[slug]`
3. **UPDATE:** Click "Sửa" từ Dashboard, chỉnh sửa và lưu
4. **DELETE:** Click "Xóa" từ Dashboard, xác nhận xóa

✅ **Kết quả mong đợi:**

- Tạo, sửa, xóa bài viết thành công
- Chỉ thấy bài published ở trang chủ
- Thấy tất cả bài (kể cả draft) ở Dashboard
- RLS ngăn user sửa/xóa bài của người khác

📝 **Bài tập 5.1:** Thêm tính năng phân trang (pagination) cho danh sách bài viết ở trang chủ.

Phần 6: Comments & Realtime (Bonus)

⏱️ *Thời lượng: ~15 phút*

Trong phần bonus này, chúng ta sẽ thêm tính năng bình luận và cập nhật realtime.

6.1. Thêm form bình luận

Tạo file `src/components/posts/comment-form.tsx`:

```tsx
'use client'
```

```
import { useState } from 'react'
import { useRouter } from 'next/navigation'
import { createClient } from '@/lib/supabase/client'

interface CommentFormProps {
  postId: string
}

export function CommentForm({ postId }: CommentFormProps) {
  const router = useRouter()
  const supabase = createClient()

  const [content, setContent] = useState('')
  const [loading, setLoading] = useState(false)
  const [error, setError] = useState<string | null>(null)

  const handleSubmit = async (e: React.FormEvent) => {
    e.preventDefault()
    setError(null)
    setLoading(true)

    try {
      const { data: { user } } = await supabase.auth.getUser()

      if (!user) {
        setError('Bạn cần đăng nhập để bình luận')
        return
      }

      const { error } = await supabase
        .from('comments')
        .insert({
          post_id: postId,
          author_id: user.id,
          content,
        })

      if (error) throw error

      setContent('')
      router.refresh()
    } catch (err: any) {
      setError(err.message || 'Có lỗi xảy ra')
    } finally {
      setLoading(false)
    }
  }

  return (
    <form onSubmit={handleSubmit} className="space-y-4">
      {error && (
        <div className="bg-red-50 text-red-500 p-3 rounded-md text-sm">
          {error}
        </div>
```

```
      )}

      <div>
        <textarea
          value={content}
          onChange={(e) => setContent(e.target.value)}
          required
          rows={3}
          className="w-full px-3 py-2 border border-gray-300 rounded-md
shadow-sm focus:outline-none focus:ring-blue-500 focus:border-blue-500"
          placeholder="Viết bình luận của bạn..."
        />
      </div>

      <button
        type="submit"
        disabled={loading || !content.trim()}
        className="px-4 py-2 bg-blue-600 text-white rounded-md hover:bg-
blue-700 disabled:opacity-50 disabled:cursor-not-allowed"
      >
        {loading ? 'Đang gửi...' : 'Gửi bình luận'}
      </button>
    </form>
  )
}
```

6.2. Hiển thị danh sách bình luận

Tạo file `src/components/posts/comment-list.tsx`:

[tsx]

```
import { Comment } from '@/types/database'

interface CommentListProps {
  comments: Comment[]
}

export function CommentList({ comments }: CommentListProps) {
  if (comments.length === 0) {
    return (
      <p className="text-gray-500 text-center py-8">
        Chưa có bình luận nào. Hãy là người đầu tiên!
      </p>
    )
  }

  return (
    <div className="space-y-4">
      {comments.map((comment) => (
        <div
          key={comment.id}
          className="bg-gray-50 p-4 rounded-lg"
```

```
        >
          <div className="flex items-center gap-2 mb-2">
            <span className="font-medium">
              {comment.profiles?.display_name || 'Ẩn danh'}
            </span>
            <span className="text-gray-400 text-sm">
              {new Date(comment.created_at).toLocaleDateString('vi-VN')}
            </span>
          </div>
          <p className="text-gray-700">{comment.content}</p>
        </div>
      ))}
    </div>
  )
}
```

6.3. Tích hợp vào trang chi tiết bài viết

Cập nhật file `src/app/posts/[slug]/page.tsx`:

[tsx]

```
import { createClient } from '@/lib/supabase/server'
import { notFound } from 'next/navigation'
import type { Metadata } from 'next'
import { CommentForm } from '@/components/posts/comment-form'
import { CommentList } from '@/components/posts/comment-list'

interface PostPageProps {
  params: { slug: string }
}

export async function generateMetadata({ params }: PostPageProps):
Promise<Metadata> {
  const supabase = await createClient()

  const { data: post } = await supabase
    .from('posts')
    .select('title, excerpt')
    .eq('slug', params.slug)
    .eq('status', 'published')
    .single()

  return {
    title: post?.title || 'Bài viết',
    description: post?.excerpt || '',
  }
}

export default async function PostPage({ params }: PostPageProps) {
  const supabase = await createClient()

  // Lấy bài viết
```

```
  const { data: post, error } = await supabase
    .from('posts')
    .select(`
      *,
      profiles (
        display_name,
        avatar_url
      )
    `)
    .eq('slug', params.slug)
    .eq('status', 'published')
    .single()

  if (error || !post) {
    notFound()
  }

  // Lấy comments
  const { data: comments } = await supabase
    .from('comments')
    .select(`
      *,
      profiles (
        display_name,
        avatar_url
      )
    `)
    .eq('post_id', post.id)
    .order('created_at', { ascending: true })

  // Kiểm tra user đã đăng nhập chưa
  const { data: { user } } = await supabase.auth.getUser()

  return (
    <main className="max-w-3xl mx-auto px-4 py-8">
      <article>
        <header className="mb-8">
          <h1 className="text-4xl font-bold mb-4">{post.title}</h1>

          <div className="flex items-center gap-4 text-gray-500">
            <span>Bởi {post.profiles?.display_name || 'Ẩn danh'}</span>
            <span>•</span>
            <time>
              {post.published_at
                ? new Date(post.published_at).toLocaleDateString('vi-
VN', {
                    year: 'numeric',
                    month: 'long',
                    day: 'numeric',
                  })
                : ''}
            </time>
          </div>
        </header>
```

```
        <div className="prose prose-lg max-w-none mb-12">
          {post.content?.split('\n').map((paragraph, index) => (
            <p key={index}>{paragraph}</p>
          ))}
        </div>
      </article>

      {/* Comments Section */}
      <section className="border-t pt-8">
        <h2 className="text-2xl font-bold mb-6">
          Bình luận ({comments?.length || 0})
        </h2>

        {user ? (
          <div className="mb-8">
            <CommentForm postId={post.id} />
          </div>
        ) : (
          <p className="text-gray-500 mb-8">
            <a href="/login" className="text-blue-600 hover:text-blue-
500">
              Đăng nhập
            </a>
            {' '}để bình luận.
          </p>
        )}

        <CommentList comments={comments || []} />
      </section>
    </main>
  )
}
```

6.4. Realtime Updates (Optional)

Để hiển thị bình luận mới ngay lập tức (không cần refresh), bạn có thể sử dụng Supabase Realtime:

Tạo file `src/components/posts/realtime-comments.tsx`:

[tsx]

```
'use client'

import { useEffect, useState } from 'react'
import { createClient } from '@/lib/supabase/client'
import { Comment } from '@/types/database'
import { CommentList } from './comment-list'

interface RealtimeCommentsProps {
  postId: string
  initialComments: Comment[]
```

```
}

export function RealtimeComments({ postId, initialComments }:
RealtimeCommentsProps) {
  const supabase = createClient()
  const [comments, setComments] = useState<Comment[]>(initialComments)

  useEffect(() => {
    // Subscribe to new comments
    const channel = supabase
      .channel(`comments:${postId}`)
      .on(
        'postgres_changes',
        {
          event: 'INSERT',
          schema: 'public',
          table: 'comments',
          filter: `post_id=eq.${postId}`,
        },
        async (payload) => {
          // Fetch the new comment with profile data
          const { data: newComment } = await supabase
            .from('comments')
            .select(`
              *,
              profiles (
                display_name,
                avatar_url
              )
            `)
            .eq('id', payload.new.id)
            .single()

          if (newComment) {
            setComments((prev) => [...prev, newComment])
          }
        }
      )
      .subscribe()

    return () => {
      supabase.removeChannel(channel)
    }
  }, [postId, supabase])

  return <CommentList comments={comments} />
}
```

⚠️ **Lưu ý:** Để sử dụng Realtime, bạn cần bật Realtime cho table `comments` trong Supabase Dashboard:

1. Vào **Database → Replication**

2. Bật replication cho table `comments`

Phần 7: Tổng kết & Bài tập

7.1. Kiến thức đã học

Trong bài thực hành này, bạn đã học được:

Chủ đề	Nội dung
BaaS Concept	Hiểu Backend-as-a-Service, so sánh các nền tảng
Supabase Architecture	PostgreSQL, Auth, Realtime, Storage, Edge Functions
Database Design	Thiết kế schema, relationships, indexes
Row Level Security	Tạo policies cho SELECT, INSERT, UPDATE, DELETE
Authentication	Email/Password, OAuth (GitHub), Session management
CRUD Operations	Tạo, đọc, cập nhật, xóa dữ liệu với Supabase Client
Next.JS Integration	Server/Client components, Middleware, Protected routes

---

7.2. Cấu trúc project hoàn chỉnh

```
simple-blog/
├── src/
│   ├── app/
│   │   ├── auth/
│   │   │   └── callback/
│   │   │       └── route.ts
│   │   ├── dashboard/
│   │   │   ├── edit/
│   │   │   │   └── [id]/
│   │   │   │       └── page.tsx
│   │   │   ├── new/
│   │   │   │   └── page.tsx
│   │   │   └── page.tsx
│   │   ├── login/
│   │   │   └── page.tsx
│   │   ├── posts/
│   │   │   └── [slug]/
│   │   │       └── page.tsx
│   │   ├── register/
│   │   │   └── page.tsx
│   │   ├── actions/
```

```
│   │   │   └── auth.ts
│   │   ├── globals.css
│   │   ├── layout.tsx
│   │   └── page.tsx
│   ├── components/
│   │   ├── auth/
│   │   │   ├── login-form.tsx
│   │   │   └── register-form.tsx
│   │   ├── dashboard/
│   │   │   ├── delete-post-button.tsx
│   │   │   ├── post-form.tsx
│   │   │   └── post-list.tsx
│   │   ├── layout/
│   │   │   └── header.tsx
│   │   └── posts/
│   │       ├── comment-form.tsx
│   │       ├── comment-list.tsx
│   │       └── realtime-comments.tsx
│   ├── lib/
│   │   └── supabase/
│   │       ├── client.ts
│   │       ├── middleware.ts
│   │       └── server.ts
│   └── types/
│       └── database.ts
├── middleware.ts
├── .env.local
├── package.json
└── ...
```

7.3. Bài tập tự làm

📝 **Bài tập 7.1 (Cơ bản):** Thêm trang Profile

- Tạo trang `/profile` cho phép user xem và chỉnh sửa profile của mình
- Cho phép cập nhật display_name và avatar_url

📝 **Bài tập 7.2 (Trung bình):** Thêm tính năng Like

- Tạo bảng `likes` với schema: `post_id`, `user_id`
- Thêm RLS policies phù hợp
- Hiển thị số lượng likes và nút like/unlike trên bài viết

📝 **Bài tập 7.3 (Nâng cao):** Upload hình ảnh

- Sử dụng Supabase Storage để upload ảnh cho bài viết
- Tạo bucket và cấu hình policies
- Cho phép chèn ảnh vào nội dung bài viết

📝 **Bài tập 7.4 (Nâng cao):** Tìm kiếm bài viết

- Implement tìm kiếm full-text với PostgreSQL
- Tạo trang `/search?q=keyword`
- Hiển thị kết quả với highlight từ khóa

7.4. Tài liệu tham khảo

Tài liệu	Link
Supabase Documentation	supabase.com/docs (https://supabase.com/docs)
Supabase Auth with Next.JS	supabase.com/docs/guides/auth/server-side/nextjs (https://supabase.com/docs/guides/auth/server-side/nextjs)
Row Level Security	supabase.com/docs/guides/database/postgres/row-level-security (https://supabase.com/docs/guides/database/postgres/row-level-security)
Supabase Storage	supabase.com/docs/guides/storage (https://supabase.com/docs/guides/storage)
Supabase Realtime	supabase.com/docs/guides/realtime (https://supabase.com/docs/guides/realtime)
Next.JS App Router	nextjs.org/docs/app (https://nextjs.org/docs/app)

7.5. Checklist hoàn thành

Trước khi kết thúc, hãy đảm bảo bạn đã:

- [ ] Tạo project Supabase và lấy credentials
- [ ] Khởi tạo project Next.JS với Supabase packages
- [ ] Tạo đầy đủ 3 bảng: `profiles`, `posts`, `comments`
- [ ] Cấu hình RLS cho tất cả bảng
- [ ] Implement authentication (Email/Password + OAuth)
- [ ] Implement CRUD cho posts
- [ ] Bảo vệ routes cần authentication
- [ ] Commit code lên GitHub

🎉 **Chúc mừng!** Bạn đã hoàn thành Bài thực hành 4 — xây dựng ứng dụng full-stack với Supabase.

Ở bài tiếp theo, chúng ta sẽ học về **Full-stack Integration** — tối ưu và nâng cao ứng dụng với các tính năng như State Management, File Upload, Middleware patterns, và Performance optimization.